



字符串和字节序列

邓擎琮
北京师范大学

序列 (sequence)

- 序列：数据元素按次序排列，可以通过索引下标访问的对象。

元组

- 元素可为不同类型，不可变类型，()为界

列表

- 元素可为不同类型，可变类型，[]为界

字符串

- 元素均为字符，不可变类型，单引号/双引号/三引号为界

字节序列

- 元素为二进制编码，可变/不可变类型，b"/b" "/b"" ""/b"""" """"为界

字符串

- 字符串是由字符组成的序列
- Python没有独立的字符数据类型，单个字符的字符串被视为一个字符
- 编码
 - 计算机采用二进制
 - 数值→二进制
 - 字符→二进制

字符编码

ASCII编码

ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 低四位		ASCII控制字符											ASCII打印字符													
		0000						0001					0010	0011		0100		0101		0110		0111				
		0						1					2	3		4		5		6		7				
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	►	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◄	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	\a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◼	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	◯	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◉	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	└	^\ ^_	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	\r	回车	29	↔	^] ^_	GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	🎵	^O	SI		移入	31	▼	^~	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Backspace 代码: DEL

字符编码

- 中文的编码
 - 至少需要两个字节
 - 不能与ASCII编码冲突
 - GB2312（6763个汉字），GBK（21886个字和图形符号）等
- 全世界有上百种语言，日本把日文编到Shift_JIS里，韩国把韩文编到Euc-kr里，各国各有各的标准，**冲突在所难免！！**



字符编码

- **Unicode**: 把所有语言都统一到一套编码里，一般采用两个字节表示一个字符。

- 新问题:

'A': 01000001 → 00000000 01000001

纯英文的存储空间比**ASCII**编码的大一倍

- **UTF-8**编码: 可变长编码，节约空间，采用**1-6**个字节表示一个字符: 英文字母**1**个字节（兼容**ASCII**），汉字通常**3**个字节，很生僻的字符**4-6**个字节。

字符	ASCII	Unicode	UTF-8
A	01000001	00000000 01000001	01000001
中	x	01001110 00101101	11100100 10111000 10101101

字符编码4

- Python 3中，字符串默认使用Unicode编码

```
>>> u'abc'  
'abc'
```

- `ord()`: 获取字符的整数表示的Unicode码
- `chr()`: 把整数表示的Unicode码转换成对应的字符

```
>>> ord('a')
```

```
97
```

```
>>> ord('文')
```

```
25991
```

```
>>> hex(97)
```

```
'0x61'
```

```
>>> hex(25991)
```

```
'0x6587'
```

```
>>> chr(97)
```

```
'a'
```

```
>>> chr(25991)
```

```
'文'
```

```
>>> print('\u4e2d\u6587\u0061\x62\143')
```

```
中文abc
```

字符串的定义

■ ''

```
>>> print('abc', type('abc'))
```

```
abc <class 'str'>
```

■ \"

```
>>> print("xyz", type("xyz"))
```

```
xyz <class 'str'>
```

■ ''' '''

```
>>> print(''a  
b c'')
```

```
a
```

```
b c
```

```
>>> print(""" x  
y z""")
```

```
x
```

```
y z
```

■ str()

```
>>> print('Blue' 'Sky')
```

```
BlueSky
```

```
>>> print(str(), type(str()))
```

```
<class 'str'>
```

```
>>> str(123)
```

```
'123'
```

```
>>> str(complex(1, 2))
```

```
'(1+2j)'
```

特殊字符

- 特殊字符：控制字符，用转义字符表示

转义字符	字符	转义字符	字符
\'	单引号	\n	换行
\"	双引号	\r	回车
\\	反斜杠	\t	水平制表符
\a	响铃	\v	垂直制表符
\b	退格	\ooo	八进制Unicode码对应的字符
\f	换页	\xhh	十六进制Unicode码对应的字符
		\ubbbb	十六进制Unicode码对应的字符

特殊字符示例

```
>>> print('E:\\Python\\2016\\Ch2')
```

```
E:\\Python\\2016\\Ch2
```

```
>>> print('x\\tyz')
```

```
x          yz
```

```
>>> print('x\\ny')
```

```
x
```

```
y
```

```
>>> print('\\u4e2d\\u6587\\u0061\\x62\\143')
```

```
中文abc
```

特殊字符2

■ 字符串本身含有引号怎么办？

- 含有单引号：用双引号括住

`"I'm a student."`

- 含有双引号：用单引号括住

`'He said, "OK."'`

■ 两者都有时怎么办？

`He said, " I'm a student."`

- 用三引号

`'''He said, " I'm a student.'''`

`"""He said, " I'm a student."""`

- 一般的方法：用\`'`或\`"`来转义

`"He said, \"I'm a student.\""`

`'He said, "I\'m a student.'`

`SyntaxError: EOL while scanning string literal`

原始字符串

- r''或R'', 其中包含的\之后的字符不进行转义

```
>>> print('c:\n\td')
```

```
c:
```

```
    d
```

```
>>> print(r'c:\n\td')
```

```
c:\n\td
```

```
>>> print(r'\')
```

```
SyntaxError: EOL while scanning string literal
```

```
>>> print(r'E:\Python\2016\Ch1\')
```

```
SyntaxError: EOL while scanning string literal
```

原始字符串

- r''或R'', 其中包含的\之后的字符不进行转义

```
>>> print('c:\n\td')
```

```
c:
```

```
    d
```

```
>>> print(r'c:\n\td')
```

```
c:\n\td
```

```
>>> print(r'E:\Python\2016\Ch1\\')
```

```
E:\Python\2016\Ch1\\
```

```
>>> print(r'E:\Python\2016\Ch1'+ '\\')
```

```
E:\Python\2016\Ch1\
```

```
>>> print('E:/Python/2016/Ch1/')
```

```
E:/Python/2016/Ch1/
```

字符串的基本操作

- 序列的通用操作：切片, len, min, max等
- str对象的常用方法：
 - 类型判断
 - 大小写转换
 - 填充、空白和对齐
 - 测试、查找和替换
 - 拆分和组合
 - 编码转换
 - 格式化

类型判断

- 类型判断方法：判断字符串是否属于某个类型，返回值为True或者False。
- 示例程序：输入检测，即检验用户输入是否符合要求，如不符合则重新输入。

```
a = input('请输入一个整数：')
while not a.isdigit():
    print('注意，输入错误！')
    a = input('请输入一个整数：')
num = int(a)
print(num)
```

请输入一个整数： a
注意，输入错误！

请输入一个整数： 2.2
注意，输入错误！

请输入一个整数： 3
3

类型判断

- `str.isalnum()`: 是否全为字母或数字
- `str.isalpha()`: 是否全字母
- `str.isnumeric()`: 是否只包含数字字符
- `str.isdecimal()`: 是否只包含十进制数字
- `str.isdigit()`: 是否全数字 (0~9)
- `str.islower()`: 是否全小写
- `str.isupper()`: 是否全大写
- `str.isspace()`: 是否只包含空白
- `str.istitle()`: 是否为标题 (各单词首字母大写)
- `str.isidentifier()`: 是否是合法标识
- `str.isprintable()`: 是否全打印字符

```
>>> 'iPhone7'.isalnum()  
True
```

```
>>> '\nab'.isprintable()  
False
```

```
>>> 'Ready Go'.istitle()  
True
```

```
>>> 'lab'.isidentifier()  
False
```

```
>>> ' '.isspace()  
True
```

```
>>> '你好'.isalpha()  
True
```

类型判断

- `str.isalnum()`: 是否全为字母或数字
- `str.isalpha()`: 是否全字母
- `str.isnumeric()`: 是否只包含数字字符
- `str.isdecimal()`: 是否只包含十进制数字
- `str.isdigit()`: 是否全数字 (0~9)
- `str.islower()`: 是否全小写
- `str.isupper()`: 是否全大写
- `str.isspace()`: 是否只包含空白
- `str.istitle()`: 是否为标题 (各单词首字母大写)
- `str.isidentifier()`: 是否是合法标识
- `str.isprintable()`: 是否全打印字符

```
>>> '123'.isdecimal()
True
>>> '(1)'.isdecimal()
False
>>> '(1)'.isdigit()
True
>>> '(1)'.isnumeric()
True
>>> '五'.isdecimal()
False
>>> '五'.isdigit()
False
>>> '五'.isnumeric()
True
```

大小写转换

- `str.lower()`: 全小写
- `str.upper()`: 全大写
- `str.swapcase()`: 大小写互换
- `str.capitalize()`: 首字母大写, 其余小写
- `str.title()`: 各单词首字母大写
- `str.casefold()`: 转换成大小写无关字符串比较的格式字符串

```
>>> 'red car'.capitalize()
'Red car'
>>> 'red car'.title()
'Red Car'
>>> 'Red Car'.swapcase()
'rED cAR'
>>> 'Red Car'.upper()
'RED CAR'
```

大小写转换

- `str.lower()`: 全小写
- `str.upper()`: 全大写
- `str.swapcase()`: 大小写互换
- `str.capitalize()`: 首字母大写, 其余小写
- `str.title()`: 各单词首字母大写
- `str.casefold()`: 转换成大小写无关字符串比较的格式字符串

```
>>> 'Red Car'.lower()  
'red car'  
>>> 'Red Car'.casefold()  
'red car'
```

```
>>> 'ß'.lower()  
'ß'  
>>> 'ß'.casefold()  
'ss'
```

填充、空白和对齐

- `str.strip([chars])`: 去掉两边空格(包括`\t, \n`), 也可指定要去除的字符列表
- `str.lstrip([chars])`: 同上, 但去掉左边的
- `str.rstrip([chars])`: 同上, 但去掉右边的
- `str.zfill(width)`: 左填充, 使用0填充至width长
- `str.center(width[, fillchar])`: 中间对齐, 两边使用填充字符 (默认空格) 填充至width长度
- `str.ljust(width[, fillchar])`: 同上, 但是左对齐
- `str.rjust(width[, fillchar])`: 同上, 但是右对齐
- `str.expandtabs([tabsize])`: 设定tab的空格数, `tabsize`默认为8

填充、空白和对齐示例

```
>>> ' 123 \t\n'.strip()  
'123'
```

```
>>> ' 123 \t\n'.strip('\n')  
' 123 \t'
```

```
>>> ' 123 \t\n'.rstrip()  
' 123'
```

```
>>> '123'.zfill(5)  
'00123'
```

填充、空白和对齐示例2

```
>>> '123'.center(5)
```

```
' 123 '
```

```
>>> '123456'.center(5)
```

```
'123456'
```

```
>>> '123'.ljust(5)
```

```
'123  '
```

```
>>> '123'.ljust(5, '*')
```

```
'123**'
```

```
>>> print('a\tb')
```

```
a          b
```

```
>>> print('a\tb'.expandtabs(16))
```

```
a          b
```

测试、查找和替换

- `str.startswith(prefix[,start[,end]])`: 是否以`prefix`开头
- `str.endswith(suffix[,start[,end]])`: 是否以`suffix`结尾
- `str.count(sub[,start[,end]])`: 返回字符串`sub`出现的次数
- `str.index(sub[,start[,end]])`: `sub`第1次出现的下标, 无则导致`ValueError`
- `str.rindex(sub[,start[,end]])`: 同上, 但是最后1次
- `str.find(sub[,start[,end]])`: `sub`第1次出现的下标, 无则返回-1
- `str.rfind(sub[,start[,end]])`: 同上, 但是最后1次出现的下标
- `str.replace(old,new[,count])`: 把`old`替换成`new`, `count`为替换次数

[start,end)

测试、查找示例

```
s1='$$$ ,get rich now!!! $$$'
```

```
>>> s1.startswith('$')  
True
```

```
>>> s1.endswith('$$$')  
True
```

```
>>> s1.count('$$$')  
2
```

```
>>> s1.index('$$$')  
0
```

```
>>> s1.rindex('$$$')  
20
```

```
>>> s1.index('$$$ ', 3, 22)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#19>", line 1, in <module>
```

```
    s1.index('$$$ ', 3, 22)
```

```
ValueError: substring not found
```

```
>>> s1.find('$$$ ', 3, 22)  
-1
```

替换示例

```
>>> s1='$$$ ,get rich now!!! $$$'
```

```
>>> s1.replace(' $$$', ' ¥')
```

```
' ¥,get rich now!!! ¥'
```

```
>>> s1
```

```
' $$$,get rich now!!! $$$'
```

```
>>> s1.replace(' $$$', ' ¥', 1)
```

```
' ¥,get rich now!!! $$$'
```

```
>>> s1.replace('hehe', ' ¥')
```

```
' $$$,get rich now!!! $$$'
```

拆分和组合

- `str.split(sep=None,maxsplit=-1)`: 按`sep`（默认为空格）分割字符串，`maxsplit`为最大分割次数（默认为无限制），返回**列表**
- `str.rsplit(sep=None,maxsplit=-1)`: 同上，但从右侧分割
- `str.splitlines([keepends])`: 按**行**分割字符串，返回**列表**
- `str.partition(sep)`: 根据`sep`分割**两部分**，返回元组(`left`, `sep`, `right`)
- `str.rpartition(sep)`: 同上，但从右侧分割
- `str.join(iterable)`: 组合`iterable`中的各元素成字符串，若包含非字符串元素，则导致**`TypeError`**

拆分示例

```
>>> '1+2+3'.split('+')      >>> '1+2+3'.rsplit('+',1)
['1', '2', '3']             ['1+2', '3']
```

```
>>> a,b,c=input('输入: ').split(',')
输入: 1,2,3
```

```
>>> '1+2+3+4'.partition('+')
('1', '+', '2+3+4')
>>> '1+2+3+4'.rpartition('+')
('1+2+3', '+', '4')
```

```
>>> 'abc\n123\nxyz'.splitlines()
['abc', '123', 'xyz']
>>> 'abc\n123\nxyz'.splitlines(True)
['abc\n', '123\n', 'xyz']
```

组合示例

```
>>> '+' .join(['1', '2', '3'])
```

```
'1+2+3'
```

```
>>> '+' .join([1, 2, 3])
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#53>", line 1, in <module>
```

```
    '+' .join([1, 2, 3])
```

```
TypeError: sequence item 0: expected str instance,
```

编码转换

- 字符串在Python内部的表示是unicode编码，因此编码转换时，需以unicode作为中间编码
- 将其他编码的字符串解码（decode）成unicode，再从unicode编码（encode）成另一种编码

B.decode(encoding='utf-8', errors='strict') -> str

str.encode(encoding='utf-8', errors='strict') -> bytes

编码转换示例

B.decode(encoding='utf-8', errors='strict') -> str
str.encode(encoding='utf-8', errors='strict') -> bytes

```
>>> '你好'.encode()
```

```
b'\xe4\xbd\xa0\xe5\xa5\xbd' ---> 字节对象
```

```
>>> '\xe4\xbd\xa0\xe5\xa5\xbd'
```

```
'ä½\xa0å¥½'
```

```
>>> b'\xe4\xbd\xa0\xe5\xa5\xbd'.decode()
```

```
'你好'
```

```
>>> '你好'.encode('unicode-escape')
```

```
b'\\u4f60\\u597d' ---> 字节对象
```

```
>>> b'\\u4f60\\u597d'.decode('unicode-escape')
```

```
'你好'
```

```
>>> '\u4f60\u597d'
```

```
'你好'
```

编码转换示例-续

B.decode(encoding='utf-8', errors='strict') -> str
str.encode(encoding='utf-8', errors='strict') -> bytes

```
>>> '你好'.encode('cp936')
```

```
b'\xc4\xe3\xba\xc3'
```

```
>>> '\xc4\xe3\xba\xc3'
```

```
'Äã°Ã'
```

```
>>> b'\xc4\xe3\xba\xc3'.decode('cp936')
```

```
'你好'
```

格式化

格式字符串.format (值1,值2,...)

```
>>> print('{1} and {0}'.format('ham', 'eggs'))  
eggs and ham
```

```
>>> print('学生人数{0}, 平均成绩{1:5.2f}'.format(34, 85))  
学生人数34, 平均成绩85.00
```

格式化

- `print(f'')`或`print(F'')`: `str.format()` 方法的简写形式

```
In [1]: a=34
```

```
In [2]: b=85
```

```
In [3]: print(f'人数{a},分数{b}')
```

人数34,分数85

```
In [4]: print(F'人数{a},分数{b:5.2f}')
```

人数34,分数85.00

```
In [5]: print('人数{0},分数{1:5.2f}'.format(a,b))
```

人数34,分数85.00

字节序列

- **bytes**和**bytearray**: 由元素为二进制编码组成的序列数据类型
- **bytes**不可变, **bytearray**可变
- **bytes**的定义: `b''`、`b" "`、`b''' '''`、`b"""" """`、`bytes()`
- **bytearray**的定义: `bytearray()`

```
>>> print (b'abc', type (b'abc'))  
b'abc' <class 'bytes'>
```

```
>>> b'abc' == 'abc'  
False
```

```
>>> b'abc' == 'abc'.encode('unicode-escape')  
True
```

```
>>> b'中'
```

```
SyntaxError: bytes can only contain ASCII literal characters.
```

字节序列2

```
>>> bytes('中',encoding='utf-8')  
b'\xe4\xb8\xad'
```

相当于:

```
>>> '中'.encode('utf-8')  
b'\xe4\xb8\xad'
```

```
>>> bytes((1,2,3))  
b'\x01\x02\x03'
```

```
>>> bytes([123,456])  
Traceback (most recent call last):  
  File "<pyshell#157>", line 1, in <module>  
    bytes([123,456])  
ValueError: bytes must be in range(0, 256)
```

字节序列3

```
>>> bytearray(b'abc')  
bytearray(b'abc')
```

```
>>> bytearray('中', 'utf-8')  
bytearray(b'\xe4\xb8\xad')
```

```
>>> bytearray((1, 2, 3))  
bytearray(b'\x01\x02\x03')
```

```
>>> bytearray((123, 456))                                     标准库struct  
Traceback (most recent call last):  
  File "<pyshell#163>", line 1, in <module>  
    bytearray((123, 456))  
ValueError: byte must be in range(0, 256)
```

Bytes和bytearray的区别

- bytes不可变， bytearray可变

```
>>> b'abc'[0]=1
```

```
Traceback (most recent call last):
```

```
File "<pyshell#129>", line 1, in <module>
```

```
b'abc'[0]=1
```

```
TypeError: 'bytes' object does not support item assignment
```

```
>>> a=bytearray(b'abc')
```

```
>>> a[0]=ord('1')
```

```
>>> a
```

```
bytearray(b'1bc')
```

字节序列的基本操作

■ 序列的通用操作

```
>>> b1=b'abc'
>>> print(len(b1),min(b1),max(b1),b1*2)
3 97 99 b'abcabc'
```

■ str对象的类似方法

```
>>> b1.replace(b'a',b'f')
b'fbc'
>>> b1.find(b'a')
0
```

■ 解码操作

```
>>> b'\xe5\xa5\xbd\xe5\xa5\xbd\xe5\xad\xa6\xe4\xb9\xa0'.decode()
'好好学习'
```